

Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales

Síntesis de Redes Activas

Trabajo Práctico de Laboratorio N° 4:

Filtros Activos

Nombre	DNI
Mangín, Matías Eduardo	35.596.112
Tardón, Damián David	36.733.088
Tito, Ricardo Clemente	35.308.739

Profesor Titular: Dr. Ing. Ferreyra Pablo
Profesor Adjunto: Ing. Reale César

Córdoba, República Argentina

2025

Índice

1. Introducción.	2
2. Desarrollo	3
2.1. Aproximación de función de transferencia por Python.	3
2.2. Análisis Topológico.	4
2.2.1. Realimentación Positiva (Sallen-Key)	4
2.2.2. Realimentación Negativa	5
2.3. Definición de Componentes	6
2.3.1. Realimentación Positiva	6
2.3.1.1. Realimentación Negativa	8
2.3.1.2. Filtro Final	9
2.4. Sensibilidad	12
2.4.1. Realimentación Positiva	12
2.4.2. Realimentación Positiva	13
2.5. Análisis de Tolerancia (Worst Case Analysis)	15
2.5.0.1. Realimentación Positiva	15
2.5.0.2. Realimentación Negativa	15
2.6. Análisis de Desviaciones - Método de Monte Carlo	16
2.7. Conclusiones	17
3. Anexo	18

1. Introducción.

El diseño de filtros activos es una etapa fundamental en el procesamiento de señales, donde los requerimientos de atenuación y selectividad demandan un enfoque sistemático y riguroso. En este informe, se presenta el desarrollo completo de un filtro activo basado en especificaciones dadas, abordando desde la aproximación teórica de la función de atenuación hasta la validación experimental del circuito implementado.

El proceso comenzó con la aproximación de la función de atenuación mediante polinomios de Chebychev, utilizando herramientas computacionales para garantizar que el diseño cumpliera con las exigencias de respuesta en frecuencia. A continuación, se sintetizó el circuito empleando topologías bicuadráticas, seleccionando entre realimentación positiva o negativa según los criterios de estabilidad y desempeño. Cada etapa del filtro fue simulada individualmente y en conjunto, utilizando LTSpice para verificar su comportamiento frente a las expectativas teóricas.

Finalmente, se analizó la sensibilidad de parámetros críticos como la frecuencia del polo (ω_p) y el ancho de banda (ω_p/Q_p), evaluando cómo las variaciones en los componentes afectan el rendimiento del filtro. Este estudio se complementó con un análisis de tolerancias, donde se determinó la peor desviación posible considerando componentes con un 10 % de variación, y se realizó una simulación de Montecarlo para visualizar el impacto estadístico de estas dispersiones.

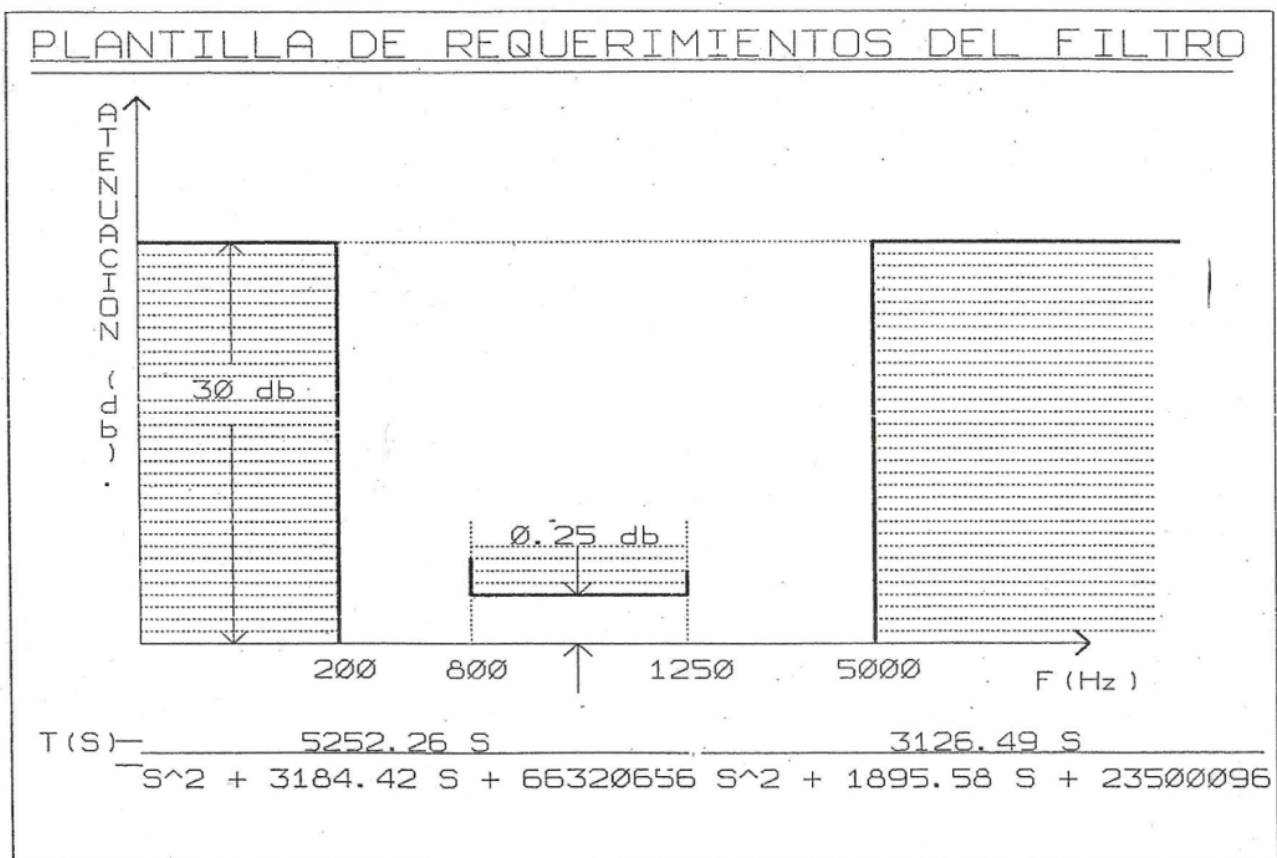


Figura 1: Plantilla de requerimientos del filtro.

2. Desarrollo

Partiendo de la planilla de requerimientos de la Fig. 1. Se sintetizó un circuito basado en amplificadores operacionales que satisfacen dichas especificaciones.

2.1. Aproximación de función de transferencia por Python.

Aproximar a la función de atenuación mediante polinomios de Chebyshev, utilizando Python. La plantilla proporciona la siguiente información: Frecuencias de interés:

- Banda de Rechazo: $200 \text{ Hz} < f_p < 5000 \text{ Hz}$
- Banda de Paso: $800 \text{ Hz} < f_r < 1250 \text{ Hz}$
- Atenuación en la Banda de Paso: $A_p = 0,25 \text{ dB}$
- Atenuación en la Banda de Rechazo: $A_s = 30 \text{ dB}$

Utilizamos la biblioteca scipy de python para poder obtener la función de transferencia del filtro correspondiente.

```
# Calcular el orden del filtro y frecuencia normalizada
n, Wp_norm = signal.cheblord(Wp, Ws, Ap, As, analog=True)
# Diseñar el filtro Chebyshev Tipo I
num, den = signal.cheby1(n, Ap, Wp_norm, btype='bandpass', analog=True, fs=None)
# Crear la funcion de transferencia
filtro = signal.TransferFunction(num, den)
# Convertir la funcion de transferencia en secciones de segundo orden (SOS)
sos = signal.tf2sos(num, den)
```

La función *Cheb1ord*(*w_p*, *w_s*, *g_{pass}*, *g_{stop}*, *analog*, *fs*) nos devuelve el orden del filtro Chebyshev tipo I analógico de orden más bajo que no pierde más de *g_{pass}* dB en la banda de paso y tiene al menos una atenuación de *g_{stop}* dB en la banda de rechazo. Luego se descompuso la función de transferencia de orden superior en secciones bicuadráticas, usando *tf2sos*(*denominador*, *numerador*), lo que es útil para implementar filtros en hardware o software.

Secciones bicuadráticas:

Sección 1:

Numerador: [16420899.35090685 0. 0.]

Denominador: [1.00000000e+00 3.18442632e+03 6.63208202e+07]

Sección 2:

Numerador: [1. 0. 0.]

Denominador: [1.00000000e+00 1.89557535e+03 2.35000932e+07]

Para poder analizar gráficamente la función de transferencia y visualizar como se atenúa o amplía en las frecuencias deseadas o cómo afecta la fase de la señal, se generó el diagrama de Bode por código en Python, con la función *control.bode_plot*(). Resultando en el siguiente gráfico:

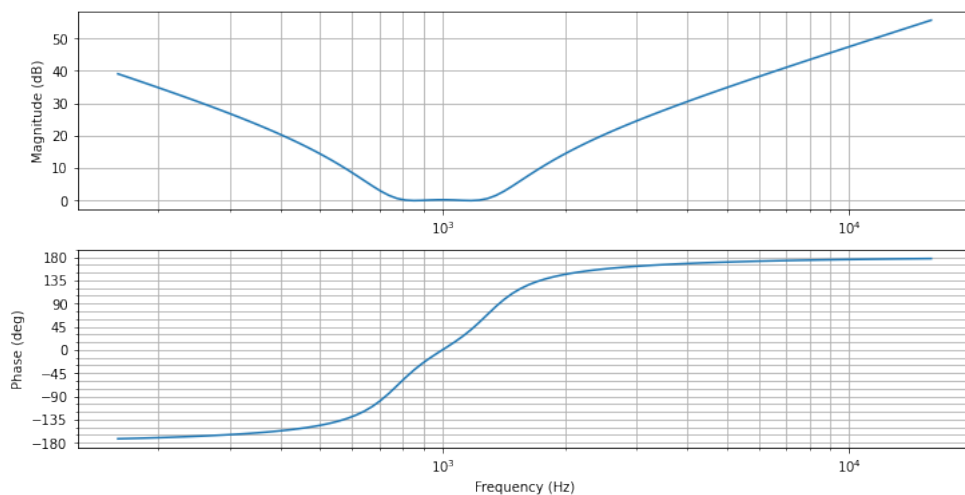


Figura 2: Magnitud y Fase de la Función de Transferencia

La Funcion de Transferencia es:

$$\frac{1.642e+07 \text{ s}^2}{s^4 + 5080 \text{ s}^3 + 9.586e+07 \text{ s}^2 + 2.006e+11 \text{ s} + 1.559e+15}$$

2.2. Análisis Topológico.

Se sintetizó un circuito que satisfaga los requerimientos del punto anterior, utilizando topologías bicuadráticas de realimentación positiva y negativa. Para poder implementar el filtro, se analizaron ambas topologías canónicas. Haciendo uso del módulo SymPy de Python, se consiguieron las funciones de transferencia para cada una.

2.2.1. Realimentación Positiva (Sallen-Key)

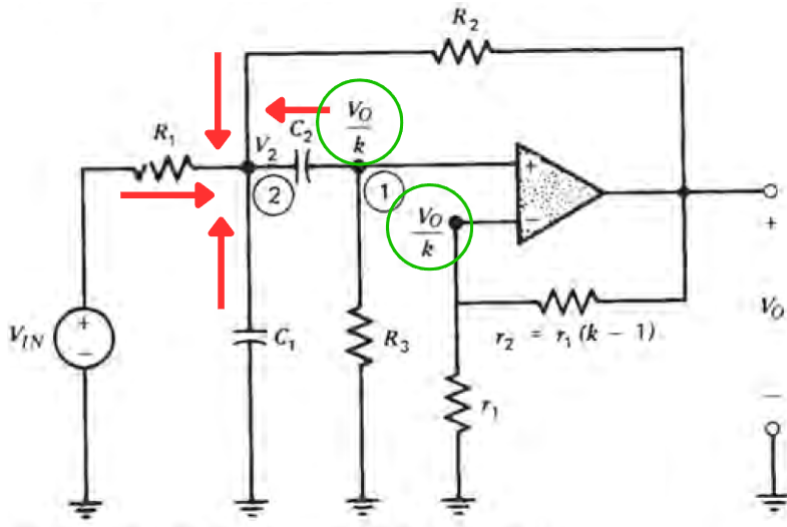


Figura 3: Topología de realimentación positiva (Sallen-Key BP).

En base a la imagen, definiendo los nodos, las corrientes entrantes y salientes (según las flechas rojas) y las caídas de tensión (en círculos verdes). Pudimos, basándonos en los parámetros definidos y usando un algoritmo en Python, obtener la función de transferencia del circuito, de la siguiente forma:

```
# Definimos las variables simbolicas
V2, Vo, Vi, C1, C2, R1, R2, R3, s, k = sp.symbols('V2_Vo_Vi_C1_C2_R1_R2_R3_s_k')
# Definimos las ecuaciones
eq1 = V2 * (C1 * s + C2 * s + 1/R2 + 1/R1) - Vo/R2 - Vi/R1 - (C2 * Vo * s)/k
eq2 = (Vo * (C2 * s + 1/R3)) / k - C2 * V2 * s
# Resolver el sistema de ecuaciones para V2 y Vo
solucion = sp.solve([eq1, eq2], (V2, Vo))
# Extraer la solucion para Vo
Vo_sol = solucion[Vo]
# Simplificar la expresion de Vo/Vi
funcion_transferencia = sp.simplify(Vo_sol / Vi)
# Manipular la funcion de transferencia para que tenga la forma deseada
# Dividimos numerador y denominador por el coeficiente de s^2 en el denominador
coeficiente_s2 = C1 * C2 * R1 * R2 * R3
# Simplificar nuevamente
funcion_transferencia = sp.simplify(funcion_transferencia/coeficiente_s2)
# Mostrar la funcion de transferencia
print("Funcion de transferencia Vo/Vi:")
sp.pretty_print(funcion_transferencia)
```

El resultado de impreso a la salida es:

$$k*s/(C1*R1*(C1*C2*R1*R2*R3*s**2 + C1*R1*R2*s + C2*R1*R2*s - C2*R1*R3*k*s + C2*R1*R3*s$$

Esta se puede reordenar para obtener una mejor aproximación a la función de transferencia deseada:

$$G(s) = \frac{k_1 s}{s^2 + a_1 s + b_1} \quad (1)$$

La función de transferencia tendrá la forma:

$$G(s) = \frac{k}{C_1 R_1} \cdot \frac{s}{s^2 + s \left(\frac{1}{C_2 R_3} + \frac{1}{C_1 R_3} - \frac{k}{C_1 R_2} + \frac{1}{C_1 R_2} + \frac{1}{C_1 R_1} \right) + \frac{1}{C_1 C_2 R_2 R_3} + \frac{1}{C_1 C_2 R_1 R_3}} \quad (2)$$

2.2.2. Realimentación Negativa

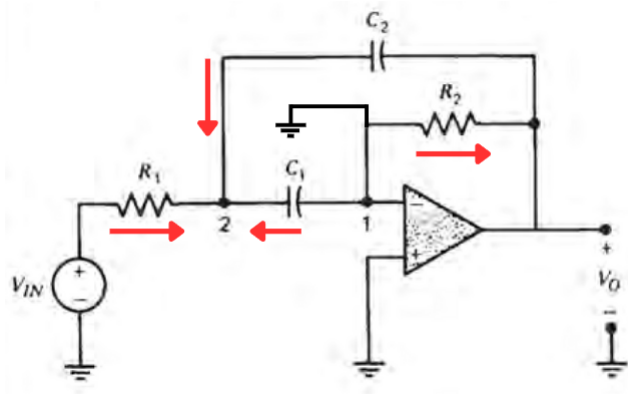


Figura 4: Topología de realimentación negativa (BP).

Al igual que el circuito de realimentación positiva, se definió la dirección de las corrientes en los nodos y las caídas de tensión. Para luego usar un algoritmo en Python para la carga matemática y obtener la función de transferencia:

```
# Definimos las variables simbolicas
V2, Vo, Vi, C1, C2, R1, R2, R3, s, k = sp.symbols('V2 Vo Vi C1 C2 R1 R2 R3 s k')
# Definimos las ecuaciones
eq1 = V2 * (C1 * s + C2 * s + 1/R1) - Vi/R1 - C2 * Vo * s
eq2 = -C1 * V2 * s - Vo/R2
# Resolver el sistema de ecuaciones para V2 y Vo
solucion = sp.solve([eq1, eq2], (V2, Vo))
# Extraer la solucion para Vo
Vo_sol = solucion[Vo]
# Simplificar la expresion de Vo/Vi
funcion_transferencia = sp.simplify(Vo_sol / Vi)
# Manipular la funcion de transferencia para que tenga la forma deseada
# Dividimos numerador y denominador por el coeficiente de s^2 en el denominador
coeficiente_s2 = C1 * R2
funcion_transferencia = sp.simplify(funcion_transferencia/coeficiente_s2)
print(funcion_transferencia)
# Mostrar la funcion de transferencia
print("funcion_de_transferencia Vo/Vi:")
sp.pretty_print(funcion_transferencia)
```

Resultando en la siguiente Función de Transferencia:

$$G = \frac{-s}{C_1 C_2 R_1 R_2 s^2 + C_1 R_1 s + C_2 R_1 s + 1}$$

La función de transferencia tiene la forma:

$$G = -\frac{\frac{1}{C_2 R_1} s}{s^2 + s \left(\frac{1}{C_2 R_2} + \frac{1}{C_1 R_2} \right) + \frac{1}{C_1 C_2 R_1 R_2}} = \frac{k_2 s}{s^2 + a_2 s + b_2}$$

2.3. Definición de Componentes

2.3.1. Realimentación Positiva

En el diseño de filtros activos, **Daryanani** en **Principles of Active Network Synthesis And Design** propone un método para simplificar el diseño al reducir el número de incógnitas. En este caso, se asumen condiciones como:

$$C_1 = C_2 = 1 \quad y \quad R_1 = R_2 = R_3 = R$$

Esto reduce el número de variables y permite resolver el sistema de ecuaciones de manera más sencilla. Estas suposiciones son comunes en el diseño de filtros y se basan en la normalización de componentes.

Las ecuaciones de diseño proporcionadas son:

Función de transferencia:

$$G = \frac{k_1 s}{s^2 + a_1 s + b_1} = \frac{5252,26s}{s^2 + 3184,42s + 66320656}$$

Relaciones entre parámetros:

$$\begin{aligned} k_1 &= \frac{k}{C_1 R_1} \\ a_1 &= \frac{1}{C_2 R_3} + \frac{1}{C_1 R_3} - \frac{k}{C_1 R_2} + \frac{1}{C_1 R_2} + \frac{1}{C_1 R_1} \\ b_1 &= \frac{1}{C_1 C_2 R_2 R_3} + \frac{1}{C_1 C_2 R_1 R_3} \end{aligned}$$

Condiciones de Daryanani:

$$C_1 = C_2 = 1 \quad y \quad R_1 = R_2 = R_3 = R$$

Ecuaciones simplificadas:

$$\begin{aligned} -\frac{k}{R} + \frac{4}{R} &= 3184,42 \\ \frac{2}{R^2} &= 66320656 \end{aligned}$$

Para poder obtener el valor correspondiente a la resistencia R y C, según la simplificación ofrecida por **Daryanani**, se aplicó el siguiente algoritmo:

```
# Definir las variables simbolicas
k, R, C = sp.symbols('k R C')
C = 100e-9 # defino a C en 100 nF
# Ecuaciones simplificadas
eq1 = -k/(C*R) + 4/(C*R) - 3184.42
eq2 = 2/(C**2*R**2) - 66320656
# Resolver el sistema de ecuaciones
R = sp.solve(eq2, R) # defino el valor de R
R = R[1]
eq1 = -k/(C*R) + 4/(C*R) - 3184.42 # redefino la ecuacion para el nuevo valor de R
K, = sp.solve(eq1, k) # como es cuadratica obtengo un valor positivo y otro negativo
```

Generando el siguiente resultado:

```
#Solucion del sistema de ecuaciones:
k = 3.44700534446283
R = 1736.56319058784
```

Pudimos aproximar a estos resultados implementando las siguientes resistencias comerciales:

$$\begin{aligned} R &= 1,5 \text{ k}\Omega + 220 \text{ }\Omega + 18 \text{ }\Omega \\ k &\approx 3,45 = 1 + \frac{r_2}{r_1} \\ r_1 &= 1 \text{ k}\Omega \quad y \quad r_2 = 2,44 \text{ k}\Omega \\ r_2 &= 2,2 \text{ k}\Omega + 720 \text{ }\Omega // 720 \text{ }\Omega // 720 \text{ }\Omega \end{aligned}$$

Analizando a cada resistencia, sin perder precisión, ya que las tolerancias de los componentes garantizan que el valor real esté dentro del rango aceptable. Esto facilitó la selección de componentes y el diseño del circuito, redefiniendo los valores:

$$R = 1,8 \text{ k}\Omega$$

$$r_1 = 1 \text{ k}\Omega$$

$$r_2 = 2,2 \text{ k}\Omega + 220 \text{ }\Omega$$

$$C_1 = 100 \text{ nF}$$

$$C_2 = 100 \text{ nF}$$

Definido el circuito y sus componentes, se construyó el circuito en el simulador LTSpice. Se usó una fuente (E1) como medio de simulación de la función de transferencia del circuito deseado, para comparar la salida requerida (Vout1) y la salida obtenida del circuito diseñado (Vout). El circuito se muestra de la siguiente imagen:

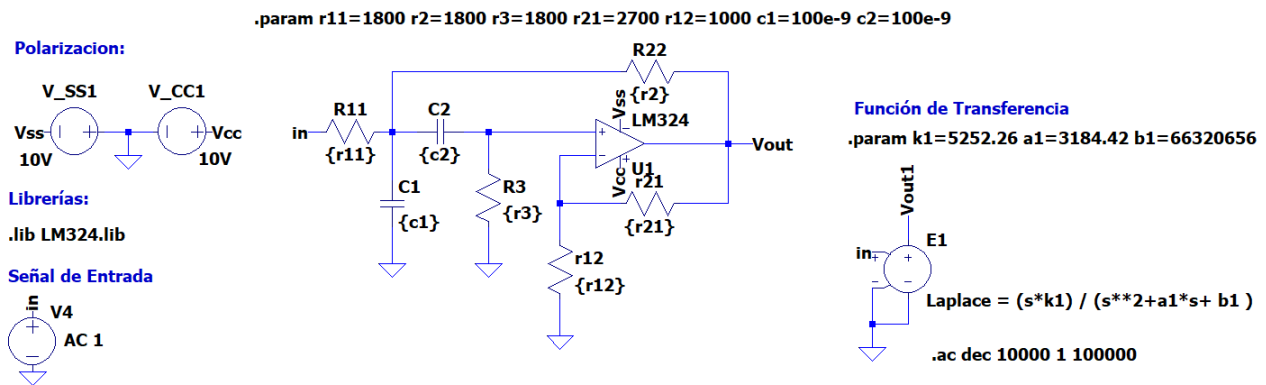


Figura 5: Circuito Simulado en LTSpice

Así mismo se compararon los diagramas de Bode de ambas salidas, mostrando una respuesta en frecuencia de magnitud y fase similares:

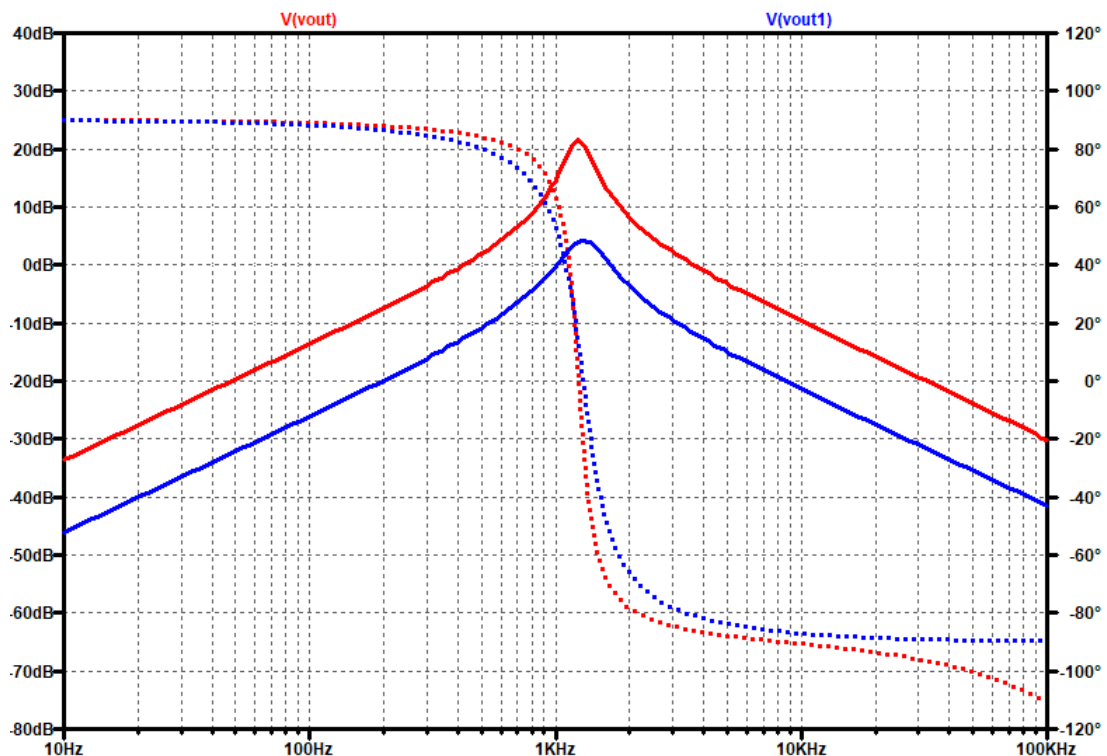


Figura 6: Magnitud y Fase: La Función de Transferencia vs Salida del Circuito

Considerando la diferencia entre ambas respuestas en frecuencia como aceptables se procedió a definir los valores de la realimentación negativa.

2.3.1.1. Realimentación Negativa

Siguiente a la bibliografía correspondiente **Daryanani** en **Principles of Active Network Synthesis And Design**, se asumen condiciones como:

$$C_1 = C_2 = 1 \quad y \quad R_1 = R_2 = R_3 = R$$

Las ecuaciones de diseño proporcionadas son:

Función de transferencia:

$$G = \frac{k_1 s}{s^2 + a_1 s + b_1} = \frac{3126,49s}{s^2 + 1895,58s + 203500096}$$

Relaciones entre parámetros:

$$\begin{aligned} k_2 &= -\frac{k}{C_2 R_1} \\ a_2 &= \frac{1}{C_2 R_2} + \frac{k}{C_1 R_2} \\ b_2 &= \frac{1}{C_1 C_2 R_1 R_2} \end{aligned}$$

Condiciones de Daryanani:

$$C_1 = C_2 = 1$$

Ecuaciones simplificadas:

$$\begin{aligned} \frac{2}{R_2} &= 1895,58 \\ \frac{1}{R_1 R_2} &= 23500096 \end{aligned}$$

Para poder obtener el valor correspondiente a la resistencia R y C, según la simplificación ofrecida por Daryanani, se aplicó un algoritmo según las ecuaciones desarrolladas anteriormente. Generando el siguiente resultado:

Solución del sistema de ecuaciones Realimentación Positiva:

R1=403.313246039506

R2=10550.8604226675

C1=1e-07

C2=1e-07

Pudimos aproximar a estos resultados implementando las siguientes resistencias comerciales:

$$\begin{aligned} R_1 &= 400 \, \Omega \\ k \approx 3,45 &= 1 + \frac{r_2}{r_1} \\ r_1 &= 1 \, k\Omega \quad y \quad r_2 = 2,44 \, k\Omega \\ r_2 &= 2,2 \, k\Omega + 720 \, \Omega // 720 \, \Omega // 720 \, \Omega \end{aligned}$$

Analizando a cada resistencia, sin perder precisión, ya que las tolerancias de los componentes garantizan que el valor real esté dentro del rango aceptable. Esto facilitó la selección de componentes y el diseño del circuito, redefiniendo los valores:

$$\begin{aligned} R &= 1,8 \, k\Omega \\ r_1 &= 1 \, k\Omega \\ r_2 &= 2,2 \, k\Omega + 220 \, \Omega \\ C_1 &= 100 \, nF \\ C_2 &= 100 \, nF \end{aligned}$$

De manera similar a la realimentación positiva, se usó una fuente (E1) para poder generar la salida, según la función de transferencia deseada. Con la finalidad de comparar la salida con el circuito diseñado. La siguiente imagen muestra el circuito diseñado y la fuente usada para generar la salida deseada:

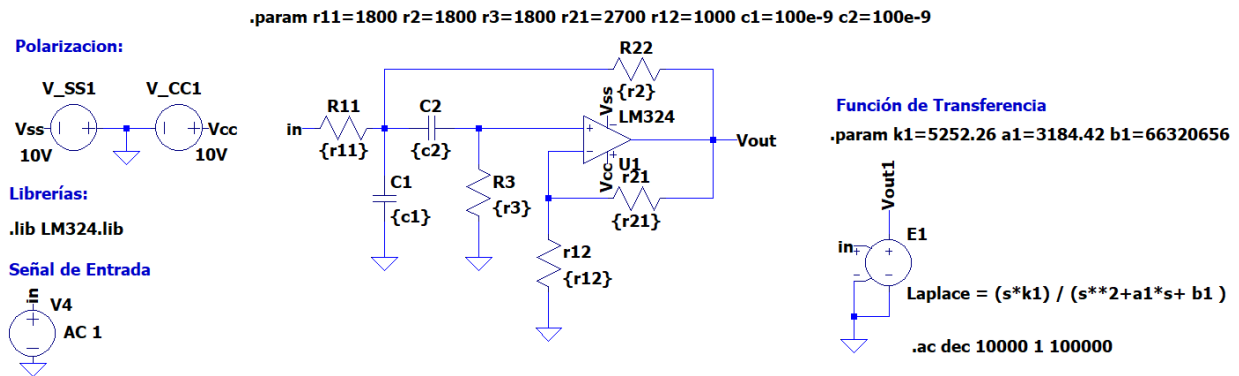


Figura 7: Circuito Simulado en LTSpice

Comparando la respuesta en frecuencia de ambas salidas, se concluyó al igual que en el anterior circuito una diferencia aceptable. Esta comparación se muestra en la siguiente imagen:

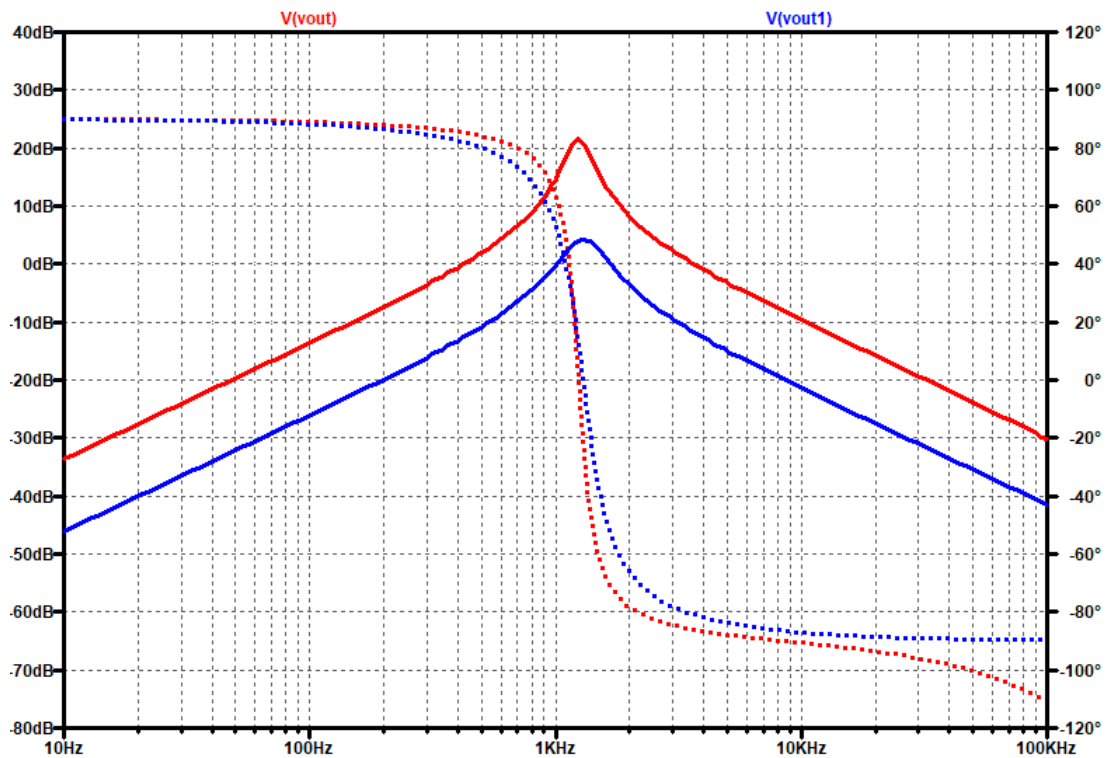


Figura 8: Magnitud y Fase: La Función de Transferencia vs Salida del Circuito

2.3.1.2. Filtro Final

Al realizar la unificación de ambos circuitos, tanto de realimentación positiva como negativa, y comparando con la salida deseada, calculada como el producto de ambas funciones de transferencias. Se compararon ambas de la siguiente manera:

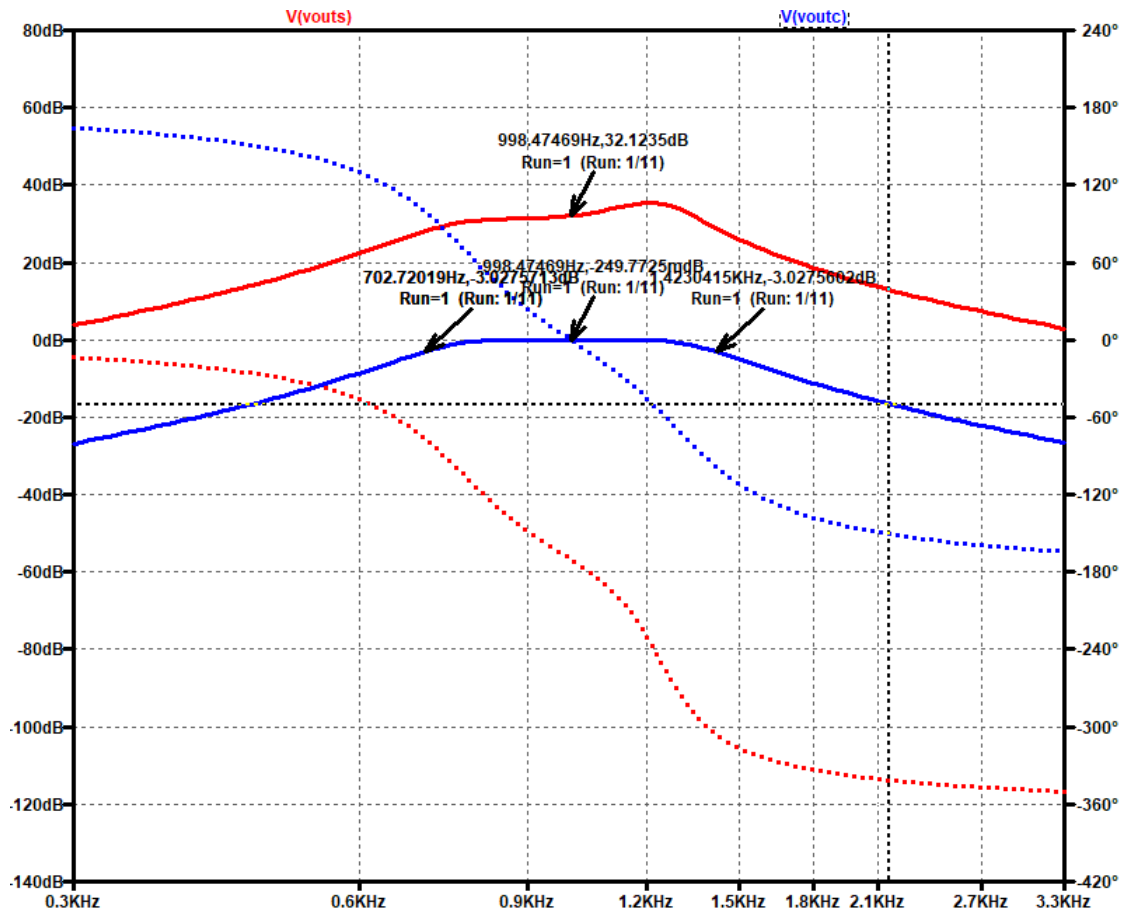


Figura 9: Magnitud y Fase: La Función de Transferencia vs Salida del Circuito

Esta diferencia sí se encontró no satisfactoria. Con lo cual se procedió a diseñar un filtro a la salida con la finalidad de reducir la diferencia de magnitud y fase, entre las dos respuestas en frecuencia. Tomando del diagrama de Bode, se observa una diferencia de aproximadamente 32 dB , con lo cual se diseñó una tercera etapa inversora con una atenuación del orden de dicha diferencia. Implementando nuevamente un algoritmo de Python para la carga matemática:

```

atenuacion=32 # [dB]
# ganancia en ganancia
att=10**(atenuacion/20)
# inversa de la atenuacion
k=1/att
# defino una de las resistencias R1 = 100 kOhm
R1=100e3
R2=R1*k

```

El Código nos permite definir las resistencias:

$$R_1 = 1\text{ k}\Omega$$

$$R_2 = 2,2\text{ k}\Omega + 220\text{ }\Omega$$

Definido el diseño, se implementó a la salida del circuito unificado, considerando que se tiene un valor aproximado ya que se basa en los valores comerciales de las resistencias:

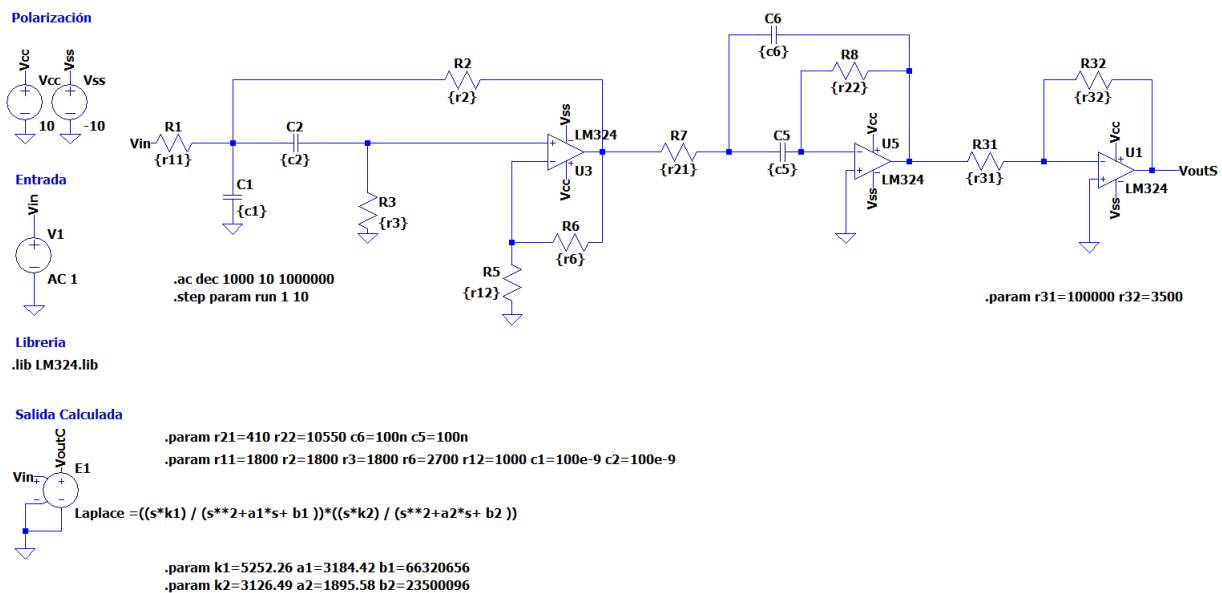


Figura 10: Circuito Final, con Filtro implementado a la salida.

Las respuestas en frecuencia comparadas, muestran la siguiente gráfica:

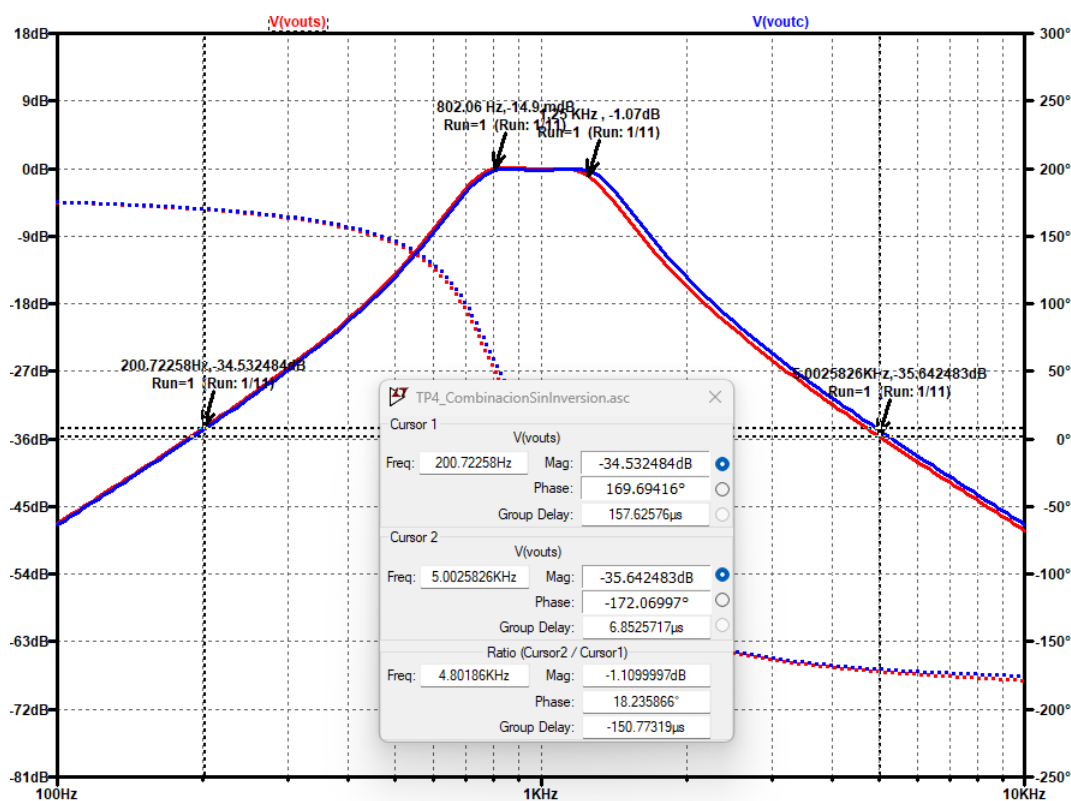


Figura 11: Magnitud y Fase: (Voutc) La Función de Transferencia Calculada comparada con la Salida del Circuito Simulada (Vouts)

Con lo que se tiene los siguientes valores: Atenuación de 14,9 dB en 800 Hz y de 1,07 dB en 1250 Hz, muy aproximada a los valores requeridos. Mientras una atenuación mayor a 34 dB y 34 dB para los 200 Hz y 5 kHz respectivamente. Por lo que se sintetizó un filtro casi idéntico al requerido.

2.4. Sensibilidad

Los valores de sensibilidad (S) cuantifican cómo cambia un parámetro del filtro ante variaciones en un componente específico. Matemáticamente, se definen como:

$$S_x^y = \frac{\frac{\partial y}{\partial x}}{\frac{y}{x}}$$

Se define a la variables $\mathbf{x}$ como aquel componente reactivo (C) o resistivo (R) a implementar, a la variable $\mathbf{y}$ como el parámetro del filtro ($\omega_p, b\omega_p, Q_p$). El valor resultante indica el cambio porcentual en y ante un cambio porcentual en x .

La función bicuadrática expresada a razón de los parámetros ω_p , ω_z , Q_p , Q_z y K es:

$$G = K \frac{s^2 + \frac{\omega_z}{Q_z} s + \omega_z^2}{s^2 + \frac{\omega_p}{Q_p} s + \omega_p^2}$$

Según las propiedades desarrolladas en el **Daryanani** para la sensibilidad:

$$\begin{aligned} S_x^{p_1 p_2} &= S_x^{p_1} + S_x^{p_2} \\ S_x^{p_1/p_2} &= S_x^{p_1} - S_x^{p_2} \\ S_x^{p_n} &= \frac{1}{n} S_x^p \\ S_x^{p^n} &= n S_x^p \\ S_x^{p_1 + p_2} &= \frac{p_1 S_x^{p_1} + p_2 S_x^{p_2}}{p_1 + p_2} \\ S_x^{cf(x)} &= S_x^{f(x)} \end{aligned}$$

Figura 12: Propiedades Sensibilidad - Gobind Daryanani, Principles of Active Network página 149

Nuevamente se usó un algoritmo de Python para la carga matemática. Haciendo uso de las propiedades, se cargó una función que se llamaría según la forma de la variable x e y .

2.4.1. Realimentación Positiva

Retomando la función de transferencia obtenida (2) e igualando los parámetros de la función bicuadrática (2.4), obtenemos las siguientes igualdades:

$$\omega_p^2 = \frac{1}{R_1 C_1 R_3 C_2} + \frac{1}{R_2 C_2 R_3 C_1} = \omega_1 \omega_{32} + \omega_2 \omega_{31}$$

Considerando una suma y las propiedades de producto. Desarrollamos la ecuación de la siguiente manera:

$$\begin{aligned} S_x^{\omega_p^2} &= \frac{\omega_1 \omega_{32} (S_x^{\omega_1} + S_x^{\omega_{32}}) + \omega_2 \omega_{31} (S_x^{\omega_2} + S_x^{\omega_{31}})}{\omega_1 \omega_{32} + \omega_2 \omega_{31}} \\ S_x^{\omega_p^2} &= 2 S_x^{\omega_p} \end{aligned}$$

Siendo x un componente cualquiera del circuito, con ello se definieron las diferentes sensibilidades en base a los componentes de interés del circuito:

$$\begin{aligned} S_{R_1}^{\omega_p} &= -\frac{1}{2} \frac{R_2}{R_1 + R_2} \\ S_{R_2}^{\omega_p} &= -\frac{1}{2} \frac{R_1}{R_1 + R_2} \\ S_{R_3}^{\omega_p} &= -\frac{1}{2} \\ S_{C_1}^{\omega_p} &= -\frac{1}{2} \\ S_{C_2}^{\omega_p} &= -\frac{1}{2} \end{aligned}$$

Los resultados correspondientes a la frecuencia del polo ω_p :

Sensibilidades para w_p :

$$\begin{aligned} S_{R1}^{w_p} &= -0.2500 \\ S_{R2}^{w_p} &= -0.2500 \\ S_{R3}^{w_p} &= -0.5000 \\ S_{C1}^{w_p} &= -0.5000 \\ S_{C2}^{w_p} &= -0.5000 \end{aligned}$$

Estas sensibilidades calculadas, no permite llegar a las siguientes conclusiones:

- Al aumentar un 1 % R_1 o R_2 disminuye ω_p un 0,25 %
- Al aumentar un 1 % R_3 , % C_1 o % C_2 disminuye $b\omega_p$ un 0,5 % demostrando una mayor sensibilidad.
- El sistema muestra una mayor sensibilidad a los cambios en los capacitores y R_3 que en las resistencia R_1 y R_2 .

Mientras el ancho de banda del polo (ω_p) se define como el cociente entre la frecuencia del polo sobre el factor de calidad (agudeza del pico de resonancia):

$$\frac{\omega_p}{Q_p} = b\omega_p = \frac{1}{R_1 C_1} (1 - k) + \frac{1}{R_1 C_1} + \frac{1}{R_3 C_1} + \frac{1}{R_3 C_2} = \omega_{21} (1 - k) + \omega_1 + \omega_{31} + \omega_{32}$$

Definiendo las sensibilidades para cada componente:

$$\begin{aligned} S_{R1}^{b\omega_p} &= -\frac{1}{b\omega_p} \frac{1}{C_1 R_1} \\ S_{R2}^{b\omega_p} &= -\frac{1}{b\omega_p} \frac{1 - k}{C_1 R_2} \\ S_{R3}^{b\omega_p} &= -\frac{1}{b\omega_p} \frac{C_1 + C_2}{C_1 C_2 R_3} \\ S_{C1}^{b\omega_p} &= -\frac{1}{b\omega_p} \frac{R_1 R_2 + R_1 R_3 (1 - k) + R_2 R_3}{C_1 R_1 R_2 R_3} \\ S_{C2}^{b\omega_p} &= -\frac{1}{b\omega_p} \frac{1}{C_2 R_3} \end{aligned}$$

Definidas las ecuaciones y los valores de cada componente. El algoritmo entrega los siguientes valores:

Sensibilidades para bw_p :

$$\begin{aligned} S_{R1}^{bw_p} &= -1.7857 \\ S_{R2}^{bw_p} &= 4.3571 \\ S_{R3}^{bw_p} &= -3.5714 \\ S_{C1}^{bw_p} &= 0.7857 \\ S_{C2}^{bw_p} &= -1.7857 \end{aligned}$$

Las sensibilidades calculadas nos permiten concluir;

- Al aumentar un 1 % R_1 disminuye $b\omega_p$ un 1,78 %
- Al aumentar un 1 % R_2 aumenta $b\omega_p$ un 4 % demostrando una mayor sensibilidad.
- Al aumentar un 1 % R_3 disminuye $b\omega_p$ un casi 3,6 %
- El sistema muestra una menor sensibilidad a los cambios en los capacitores que en las resistencia, tanto en aumento como decremento de $b\omega_p$

2.4.2. Realimentación Positiva

Retomando la función de transferencia obtenida (2.2.2) e igualando los parámetros de la función bicuadrática (2.4), obtenemos las siguientes igualdades:

$$\omega_p^2 = \frac{1}{R_1 C_1 R_2 C_2} = \omega_1 \omega_2$$

Considerando las propiedades antes aplicadas y el mismo procedimiento para obtener las diferentes sensibilidades. Se obtuvo los parámetros de interés respecto a los componentes del circuito:

$$\begin{aligned} S_{R_1}^{\omega_p} &= -\frac{1}{2} \\ S_{R_2}^{\omega_p} &= -\frac{1}{2} \\ S_{C_1}^{\omega_p} &= -\frac{1}{2} \\ S_{C_2}^{\omega_p} &= -\frac{1}{2} \end{aligned}$$

Los resultados correspondientes a la frecuencia del polo ω_p :

Sensibilidades para w_p :

$$S_{R1}^{\omega_p} = -0.5000$$

$$S_{R2}^{\omega_p} = -0.5000$$

$$S_{C1}^{\omega_p} = -0.5000$$

$$S_{C2}^{\omega_p} = -0.5000$$

Estas sensibilidades calculadas, no permite llegar a las siguientes conclusiones:

- El sistema muestra una misma sensibilidad a los cambios en los capacitores y en las resistencias.

Mientras el Ancho de banda del polo se define como el cociente entre la frecuencia del polo sobre el factor de calidad (agudeza del pico de resonancia):

$$\frac{\omega_p}{Q_p} = b\omega_p = \frac{1}{R_2 C_2} + \frac{1}{R_2 C_1} = \omega_2 + \omega_{21}$$

Definiendo las sensibilidades para cada componente:

$$\begin{aligned} S_{R_1}^{b\omega_p} &= -0 \\ S_{R_2}^{b\omega_p} &= -1 \\ S_{C_1}^{b\omega_p} &= -\frac{C_2}{C_1 + C_2} \\ S_{C_2}^{b\omega_p} &= -\frac{C_1}{C_1 + C_2} \end{aligned}$$

Definidas las ecuaciones y los valores de cada componente. El algoritmo entrega los siguientes valores:

Sensibilidades para bw_p :

$$S_{R1}^{b\omega_p} = 0$$

$$S_{R2}^{b\omega_p} = -1$$

$$S_{C1}^{b\omega_p} = -0.5000$$

$$S_{C2}^{b\omega_p} = -0.5000$$

Las sensibilidades calculadas nos permiten concluir:

- Al aumentar un 1 % R_1 no afecta o cambia $b\omega_p$.
- Al aumentar un 1 % R_2 disminuye en la misma dimensión a $b\omega_p$.
- Al aumentar un 1 % C_1 y C_2 disminuye $b\omega_p$ un casi 0,5 %
- El sistema muestra una menor sensibilidad a los cambios en la resistencia R_2 que en los capacitores o la resistencia R_1 .

2.5. Analisis de Tolerancia (Worst Case Analysis)

A partir de los valores de sensibilidad obtenidos, se puede analizar el caso en que se presente múltiples componentes (x_j) y la desviación resultante de todo el circuito, por unidad de cambio en los polos, Q , ω_z , Q_z y K . Esto es definido por **Daryanani** (página 153) con las siguientes propiedades:

$$\begin{aligned}\frac{\Delta Q_p}{Q_p} &= \sum_{j=1}^m S_{x_j}^{Q_p} V_{x_j} \\ \frac{\Delta K}{K} &= \sum_{j=1}^m S_{x_j}^K V_{x_j} \\ \frac{\Delta \omega_z}{\omega_z} &= \sum_{j=1}^m S_{x_j}^{\omega_z} V_{x_j} \\ \frac{\Delta Q_z}{Q_z} &= \sum_{j=1}^m S_{x_j}^{Q_z} V_{x_j}\end{aligned}$$

En base a ello se pudo realizar la un algoritmo en Python, considerando una tolerancia del 10% en ambas topologías.

2.5.0.1. Realimentación Positiva

A partir de la tolerancia definida por el enunciado (10%) y de poder expresar a las sensibilidades, anteriormente calculadas, como una sumatoria. Se obtuvo la siguiente ecuación para poder calcular la sensibilidad en el polo ω_p :

$$\begin{aligned}\frac{\Delta \omega_p}{\omega_p} &= \sum_{j=1}^5 S_{x_j}^{\omega_p} V_{x_j} \\ \frac{\Delta \omega_p}{\omega_p} &= \frac{\Delta x}{x} (S_{R_1}^{\omega_p} + S_{R_2}^{\omega_p} + S_{R_3}^{\omega_p} + S_{C_1}^{\omega_p} + S_{C_2}^{\omega_p})\end{aligned}$$

A si mismo podemos ver como la tolerancia afecta a los componentes en el ancho de banda del polo $b\omega_p$

$$\frac{\Delta b\omega_p}{b\omega_p} = \sum_{j=1}^5 S_{x_j}^{b\omega_p} \frac{\Delta x_j}{x_j}$$

Transcribiendo las ecuaciones al script de Python se obtuvo los siguientes valores:

```
Suma sensibilidades R para w_p: -1.0 (debe ser -1)
Suma sensibilidades C para w_p: -1.0 (debe ser -1)
Delta_w_p/w_p = -0.2
```

A si mismo con el ancho de banda:

```
Suma sensibilidades R para bw_p: -1.00000000000000013 (debe ser 0)
Suma sensibilidades C para bw_p: -1.00000000000000009 (debe ser 0)
Delta_w_p/w_p = -0.200000000000000023
```

Se pudo concluir que en la peor condición de un 10% de variación de los valores nominales de todos los componentes, tanto ω_p como $b\omega_p$ varían un 20% en ambos casos.

2.5.0.2. Realimentación Negativa

A partir de la tolerancia definida por el enunciado (10%) y de poder expresar a las sensibilidades, anteriormente calculadas, como una sumatoria. Se obtuvo la siguiente ecuación para poder calcular la sensibilidad en el polo ω_p :

$$\frac{\Delta\omega_p}{\omega_p} = \sum_{j=1}^5 S_{x_j}^{\omega_p} V_{x_j}$$

$$\frac{\Delta\omega_p}{\omega_p} = \frac{\Delta x}{x} (S_{R_1}^{\omega_p} + S_{R_2}^{\omega_p} + S_{C_1}^{\omega_p} + S_{C_2}^{\omega_p})$$

A si mismo podemos ver cómo la tolerancia afecta a los componentes en el ancho de banda del polo $b\omega_p$

$$\frac{\Delta b\omega_p}{b\omega_p} = \sum_{j=1}^5 S_{x_j}^{b\omega_p} \frac{\Delta x_j}{x_j}$$

Transcribiendo las ecuaciones al script de Python se obtuvo los siguientes valores:

Suma sensibilidades R para w_p : -1.0 (debe ser -1)
Suma sensibilidades C para w_p : -1.0 (debe ser -1)
Delta_w_p/w_p = -0.2

A si mismo con el ancho de banda:

Suma sensibilidades R para bw_p : -1 (debe ser 0)
Suma sensibilidades C para bw_p : -1.0000000000000009 (debe ser 0)
Delta_bw_p/bw_p = -0.20000000000000023

Se pudo concluir que en la peor condición de un 10% de variación de los valores nominales de todos los componentes, tanto ω_p como $b\omega_p$ varían un 20% en ambos casos.

2.6. Análisis de Desviaciones - Método de Monte Carlo

Se utilizó el análisis de Monte Carlo para evaluar cómo variaciones en los valores de los componentes afectan la respuesta del filtro, particularmente observando picos en la magnitud de la respuesta en frecuencia.

El método consiste en:

- Realizar múltiples simulaciones en LTSpice. Haciendo uso de la función $MC(valor, tolerancia)$ para poder variar al componente dentro de un rango.
- En cada iteración, variar aleatoriamente los valores de los componentes dentro de su tolerancia del 10%, entre valores positivos y negativos para cada componente.
- Calcular los parámetros resultantes del filtro (ω_p , $b\omega_p$, respuesta en frecuencia)
- Analizar estadísticamente los resultados

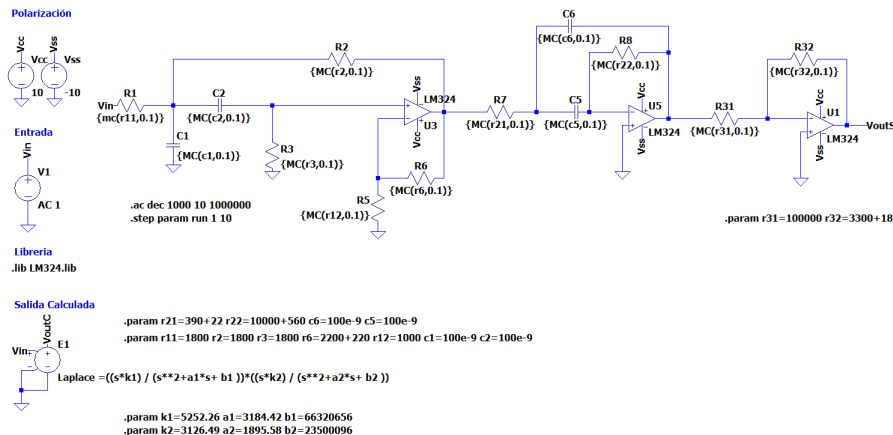


Figura 13: Circuito Final con Tolerancia en cada componente (10%)

En las curvas de respuesta en frecuencia, se obtiene una curva envolvente con las posibles respuestas, generando una más gruesa y central. La respuesta media y el ancho envolvente de la sensibilidad global del circuito.

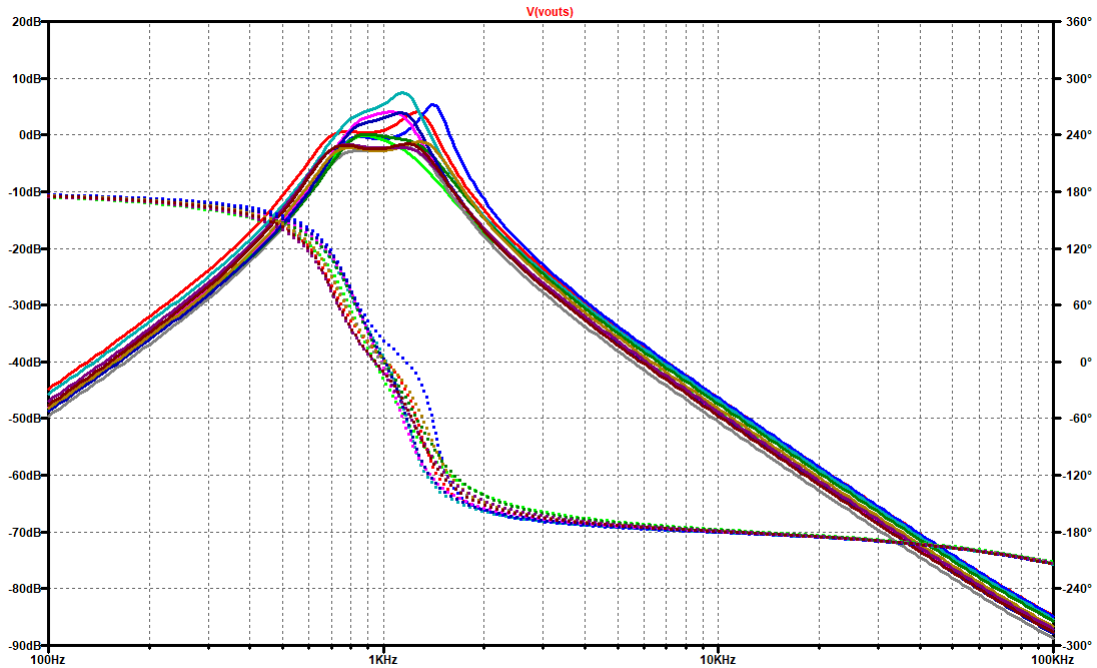


Figura 14: Respuesta de Frecuencia de las múltiples iteraciones del Método Monte Carlo

Este gráfico muestra como se podría distorsionarse la respuesta del filtro debido a la tolerancia de los componentes. Llegando a las siguientes conclusiones:

- La primera etapa de Realimentación Positiva, posee la mayor sensibilidad por poseer mayor cantidad de componentes.
- La ultima etapa es la de menor sensibilidad ya que es el que menos numero de componentes posee.
- Máxima ganancia del sistema se observa en $1,142 \text{ kHz}$ con 7 dB .

2.7. Conclusiones

A lo largo de este desarrollo, hemos abordado de manera integral el análisis, diseño y optimización de filtros activos, centrándonos en dos topologías clave: Filtro con Realimentación Positiva (Sallen-Key) y Filtro con realimentación negativa.

Identificamos que el ancho de banda ($b\omega_p$) y la frecuencia de polo (ω_p) son altamente sensibles a variaciones en componentes específicos: en el Sallen-Key, R_2 mostró una sensibilidad extrema (+4.41), lo que explica los picos abruptos en 1.2 kHz , mientras en la realimentación negativa, el comportamiento fue más estable, con sensibilidades balanceadas (± 0.5).

Por el método de Monte Carlo; Sallen-Key, mostró alta dispersión (7 dB de picos no deseados), mejorando drásticamente al reducir tolerancias al 1% , mientras en la Realimentación negativa: Variaciones más suaves (800 Hz), con menor impacto en la respuesta.

Sallen-Key: Requiere resistencias de precisión (1%) en R_2 y R_3 , y capacitores estables. La Realimentación negativa: Solo R_2 necesita ajuste fino.

3. Anexo

Links:

- Simulación Python
- Informe en Latex
- GitHub
Bibliografía